

DATA STRUCTURE

CS102

UNIT V — Sorting and Searching

Sorting Algorithms & Searching Techniques

BTechX

1. Introduction to Sorting

1.1 What is Sorting?

Sorting is the process of rearranging a sequence of elements in a specific order — either ascending or descending. It is one of the most fundamental operations in computer science.

Formally: Given a sequence of n numbers $a_1, a_2, \dots, a_n$, sorting produces a permutation $a_1', a_2', \dots, a_n'$ such that:

$a_1' \leq a_2' \leq \dots \leq a_n'$ (non-decreasing order)

□ **Key Point:** Processed/rearranged data helps in faster searching, better performance, and cleaner data management.

1.2 Sorting Example

Original list: 10, 30, 20, 80, 70, 10, 60, 40, 70

Non-decreasing: 10, 10, 20, 30, 40, 60, 70, 70, 80

Non-increasing: 80, 70, 70, 60, 40, 30, 20, 10, 10

1.3 Issues in Sorting

- How to rearrange a given set of data?
- Which data structures are most suitable to store data prior to sorting?
- How fast can sorting be achieved? (Time Complexity)
- How to sort in a memory-constrained situation? (Space Complexity)
- How to sort various types of data? (Generalization)

1.4 Categories of Sorting Algorithms

Category	Basic Idea	Examples
Sorting by Comparison	Data items are compared with each other to find the correct position.	Bubble Sort, Selection Sort, Insertion Sort, Quick Sort, Merge Sort
Sorting by Distribution	No key comparison. Items are distributed over auxiliary storage based on digits/radix, then grouped.	Radix Sort, Counting Sort, Hash-based Sort

2. Bubble Sort

2.1 How It Works

Bubble Sort is a simple comparison-based sorting algorithm. It works by repeatedly comparing and swapping adjacent elements if they are in the wrong order. In every pass, the largest unsorted element 'bubbles up' to its correct position at the end.

- In Pass 1: consider index 0 to n-1
- In Pass 2: consider index 0 to n-2
- In Pass 3: consider index 0 to n-3
- In Pass n-1: consider index 0 to 1

□ **Key Point:** In every pass, the largest element sinks to the bottom (rightmost). Total passes needed = n-1.

2.2 Step-by-Step Example

Input: 3 12 -5 6 72 21 -7 45

Pass 1:

3	12	-5	6	72	21	-7	45	
3	-5	12	6	72	21	-7	45	← swap(12, -5)
3	-5	6	12	72	21	-7	45	← swap(12, 6)
3	-5	6	12	72	21	-7	45	
3	-5	6	12	21	72	-7	45	← swap(72, 21)
3	-5	6	12	21	-7	72	45	← swap(72, -7)
3	-5	6	12	21	-7	45	72	← swap(72, 45) → 72 placed ✓

Pass 2:

```
-5  3  6 12 21 -7 45 72
-5  3  6 12 -7 21 45 72 ← swap(21, -7)
Final after Pass 2: -5  3  6 12 -7 21 45 72
```

2.3 C Code — Bubble Sort

```
void swap(int *x, int *y) {
    int tmp = *x;
    *x = *y;
    *y = tmp;
}

void bubble_sort(int x[], int n) {
    int i, j;
    for (i = n-1; i > 0; i--)
        for (j = 0; j < i; j++)
            if (x[j] > x[j+1])
                swap(&x[j], &x[j+1]);
}
```

Output Example: -45 89 -65 87 0 3 -23 19 56 21 76 -50

After sorting: -65 -50 -45 -23 0 3 19 21 56 76 87 89

2.4 Complexity Analysis

Case	Time Complexity	Reason
Best Case	$O(n)$	Array already sorted (with optimization flag)
Average Case	$O(n^2)$	Partial comparisons and swaps
Worst Case	$O(n^2)$	Array sorted in reverse order
Space Complexity	$O(1)$	In-place sorting, no extra space needed

3. Selection Sort

3.1 How It Works

Selection Sort divides the array into a sorted portion and an unsorted portion. In each pass, it finds the minimum element from the unsorted portion and swaps it with the first element of the unsorted portion.

- Step 1: Find the smallest element (mval) in $x[k \dots \text{size}-1]$
- Step 2: Swap it with $x[k]$
- Step 3: Increment k and repeat

□ **Key Point:** Selection Sort makes exactly $n-1$ swaps regardless of input — better than Bubble Sort in terms of swaps.

3.2 Step-by-Step Example

Input: -17 12 -5 6 142 21 3 45

Pass 1: Minimum = -17 (already in place) 45	→ -17 12 -5 6 142 21 3
Pass 2: Minimum = -5, swap with 12 45	→ -17 -5 12 6 142 21 3
Pass 3: Minimum = 3, swap with 12 45	→ -17 -5 3 6 142 21 12
Pass 4: Minimum = 6 (already in place) 45	→ -17 -5 3 6 142 21 12
Pass 5: Minimum = 12, swap with 142 45	→ -17 -5 3 6 12 21 142
Pass 6: Minimum = 21 (already in place) 45	→ -17 -5 3 6 12 21 142
Pass 7: Minimum = 45, swap with 142 142	→ -17 -5 3 6 12 21 45

3.3 C Code — Selection Sort

```
int findMinLoc(int x[], int k, int size) {
    int j, pos = k;
    for (j = k+1; j < size; j++)
        if (x[j] < x[pos]) pos = j;
    return pos;
}

void selectionSort(int x[], int size) {
    int k, m, temp;
    for (k = 0; k < size-1; k++) {
        m = findMinLoc(x, k, size);
        temp = x[k];
        x[k] = x[m];
        x[m] = temp;
    }
}
```

3.4 Complexity Analysis

Case	Time Complexity	Space Complexity
Best Case	$O(n^2)$	$O(1)$
Average Case	$O(n^2)$	$O(1)$
Worst Case	$O(n^2)$	$O(1)$

4. Insertion Sort

4.1 How It Works

Insertion Sort builds the sorted array one element at a time. It picks each element and inserts it into its correct position in the already-sorted portion of the array (to the left).

- Pick the element at index i (starting from $i=1$)
- Compare it with elements to its left
- Shift elements that are greater than the picked element one position to the right
- Insert the picked element at the correct position

□ **Key Point:** Insertion Sort is like sorting a hand of playing cards — you pick one card at a time and insert it in the right place.

4.2 Step-by-Step Example

Input: 54 26 93 17 77 31 44 55 20

```
[54] 26 93 17 77 31 44 55 20 ← 54 is sorted list of 1
[26 54] 93 17 77 31 44 55 20 ← inserted 26
[26 54 93] 17 77 31 44 55 20 ← inserted 93
[17 26 54 93] 77 31 44 55 20 ← inserted 17
[17 26 54 77 93] 31 44 55 20 ← inserted 77
[17 26 31 54 77 93] 44 55 20 ← inserted 31
[17 26 31 44 54 77 93] 55 20 ← inserted 44
[17 26 31 44 54 55 77 93] 20 ← inserted 55
[17 20 26 31 44 54 55 77 93] ← inserted 20 ✓
```

4.3 C Code — Insertion Sort

```
void insertionSort(int list[], int size) {
    int i, j, item;
    for (i = 1; i < size; i++) {
        item = list[i];
        for (j = i-1; (j >= 0) && (list[j] > item); j--)
            list[j+1] = list[j];
    }
}
```

```

        list[j+1] = item;
    }
}

```

4.4 Complexity Analysis

Case	Time Complexity	Reason
Best Case	$O(n)$	Already sorted array
Average Case	$O(n^2)$	Partial comparisons and shifts
Worst Case	$O(n^2)$	Reverse sorted array
Space Complexity	$O(1)$	In-place, no extra memory

5. Quick Sort

5.1 How It Works

Quick Sort is a divide-and-conquer sorting algorithm. It selects a pivot element and partitions the array into two sub-arrays: elements less than or equal to the pivot (left), and elements greater than the pivot (right). It then recursively sorts both sub-arrays.

- Step 1: Select a pivot (usually first element)
- Step 2: Partition array — smaller elements to the left, larger to the right
- Step 3: Recursively apply Quick Sort to left and right sub-arrays
- Step 4: Combine — no explicit merge needed!

□ **Key Point:** Quick Sort is generally the fastest in practice with average time complexity $O(n \log n)$. It is an in-place algorithm.

5.2 Partition Logic

Given pivot = $a[l]$ (leftmost), two pointers i and j move inward:

- i moves right until it finds an element greater than pivot
- j moves left until it finds an element less than or equal to pivot
- If $i < j$: swap $a[i]$ and $a[j]$, then continue
- If $i \geq j$: swap pivot with $a[j]$ — pivot is now in its final position

5.3 Step-by-Step Example

Input: 45 -56 78 90 -3 -6 123 0 -3 45 69 68

```

Pivot = 45:
→ Left part:  -6  -56  -3   0  -3
→ Pivot 45 placed at correct position
→ Right part: 123  90  78  45  69  68

Recursively sort left:  -56  -6  -3  -3  0
Recursively sort right: 45  68  69  78  90  123

Final: -56  -6  -3  -3  0  45  45  68  69  78  90  123

```

5.4 C Code — Quick Sort

```

int partition(int a[], int l, int r) {
    int pivot = a[l], i = l, j = r+1, t;
    while (1) {
        do { ++i; } while (a[i] <= pivot && i <= r);
        do { --j; } while (a[j] > pivot);
        if (i >= j) break;
        t = a[i]; a[i] = a[j]; a[j] = t;
    }
    t = a[l]; a[l] = a[j]; a[j] = t;
    return j;
}

void quickSort(int a[], int l, int r) {
    if (l < r) {
        int j = partition(a, l, r);
        quickSort(a, l, j-1);
        quickSort(a, j+1, r);
    }
}

```

5.5 Complexity Analysis

Case	Time Complexity	Reason
Best Case	$O(n \log n)$	Pivot always divides array into equal halves
Average Case	$O(n \log n)$	Random pivot selection
Worst Case	$O(n^2)$	Sorted/reverse-sorted array (bad pivot choice)
Space Complexity	$O(\log n)$	Recursive call stack

6. Merge Sort

6.1 How It Works

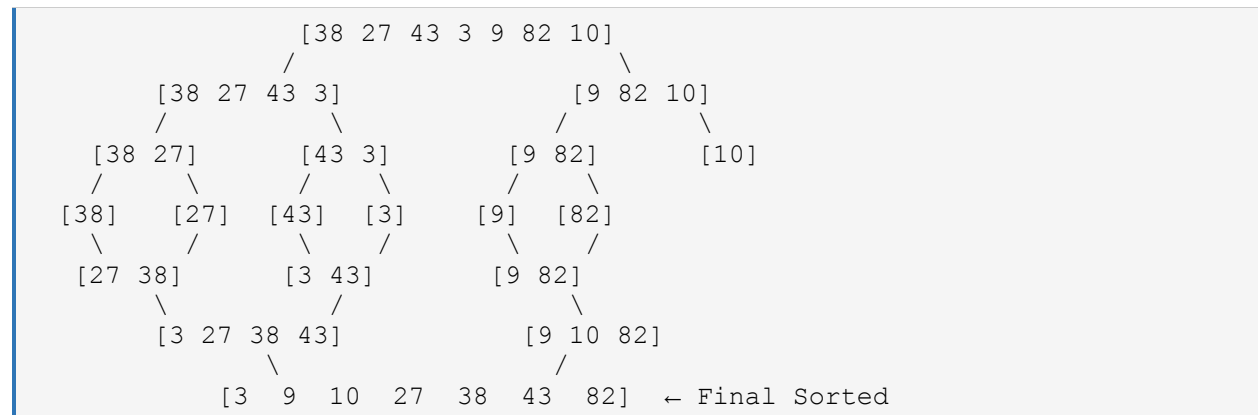
Merge Sort is a classic divide-and-conquer algorithm credited to John von Neumann. It recursively splits the array into halves until each sub-array has 1 element, then merges them back in sorted order.

- Step 1: If the list has 0 or 1 element — it is already sorted (base case)
- Step 2: If the list has more than 1 element — split it into two halves
- Step 3: Recursively apply Merge Sort to each half
- Step 4: Merge the two sorted halves into a single sorted list

□ **Key Point:** Merge Sort guarantees $O(n \log n)$ time in ALL cases — best, average, and worst. It is a stable sort.

6.2 Visual Tree (Example)

Input: 38 27 43 3 9 82 10



6.3 Merge Operation

The core of Merge Sort is the merge step. Given two sorted arrays L and R, we compare their front elements and pick the smaller one into the result array repeatedly.

```

void merge(int arr[], int l, int m, int r) {
    int n1 = m - l + 1, n2 = r - m;
    int L[n1], R[n2];
    for (int i = 0; i < n1; i++) L[i] = arr[l + i];
    for (int j = 0; j < n2; j++) R[j] = arr[m + 1 + j];
    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }
    while (i < n1) arr[k++] = L[i++];
}
  
```

```

        while (j < n2) arr[k++] = R[j++];
    }

    void mergeSort(int arr[], int l, int r) {
        if (l < r) {
            int m = (l + r) / 2;
            mergeSort(arr, l, m);
            mergeSort(arr, m+1, r);
            merge(arr, l, m, r);
        }
    }
}

```

6.4 Complexity Analysis

Case	Time Complexity	Space Complexity
Best Case	$O(n \log n)$	$O(n)$
Average Case	$O(n \log n)$	$O(n)$
Worst Case	$O(n \log n)$	$O(n)$

❑ **Note:** Unlike Quick Sort, Merge Sort requires $O(n)$ extra space for the temporary arrays. But it is stable and guaranteed $O(n \log n)$ always.

7. Comparison of Sorting Algorithms

Algorithm	Best	Average	Worst	Space	Stable?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes

❑ **Note:** A sorting algorithm is 'stable' if it preserves the relative order of equal elements.

8. Searching Algorithms

8.1 What is Searching?

Searching is the process of finding a specific element (called the key) in a collection of data. The result is either the position (index) of the element, or an indication that it was not found.

Type	Description	Pre-requisite
Linear Search	Check each element one by one from start to end until key is found.	None — works on any array
Binary Search	Divide and conquer. Compare key with the middle element and reduce the search range by half each time.	Array must be sorted

8.2 Linear Search

Linear Search (also called Sequential Search) scans through the array element by element until the target is found or the end is reached.

□ **Note:** Input: Array A[] of size n, Search key K

```

Step 1: Start from index i = 0
Step 2: Compare A[i] with K
        If A[i] == K → return i (found at position i)
        If A[i] != K → set i = i + 1
Step 3: Repeat Step 2 while i < n
Step 4: If not found → return -1

```

```

int linearSearch(int arr[], int n, int key) {
    for (int i = 0; i < n; i++)
        if (arr[i] == key) return i;
    return -1;
}

```

Case	Time Complexity
Best Case	O(1) — Key found at first position
Average Case	O(n) — Key found somewhere in middle
Worst Case	O(n) — Key not found or at last position

8.3 Binary Search

Binary Search works on a sorted array. It repeatedly divides the search interval in half. If the key equals the middle element, it's found. If the key is smaller, search the left half; if larger, search the right half.

□ **Note:** Input: Sorted Array $A[]$, Low = 0, High = $n-1$, Search Key K

Step 1: Set LOW = 0, HIGH = $n-1$

Step 2: While LOW <= HIGH:

Set MID = (LOW + HIGH) / 2

If $A[MID] == K \rightarrow$ return MID (found)

If $A[MID] < K \rightarrow$ Set LOW = MID + 1 (search right half)

If $A[MID] > K \rightarrow$ Set HIGH = MID - 1 (search left half)

Step 3: If LOW > HIGH $\rightarrow$ Key not found, return -1

```
int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (arr[mid] == key) return mid;
        else if (arr[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}
```

Binary Search Example: Search key = 40 in sorted array [10, 20, 30, 40, 50, 60, 70]

Step 1: LOW=0, HIGH=6 $\rightarrow$ MID=3 $\rightarrow A[3]=40 == 40 \rightarrow$ FOUND at index 3 ✓

Search key = 25:

Step 1: LOW=0, HIGH=6 $\rightarrow$ MID=3 $\rightarrow A[3]=40 > 25 \rightarrow$ HIGH = 2

Step 2: LOW=0, HIGH=2 $\rightarrow$ MID=1 $\rightarrow A[1]=20 < 25 \rightarrow$ LOW = 2

Step 3: LOW=2, HIGH=2 $\rightarrow$ MID=2 $\rightarrow A[2]=30 > 25 \rightarrow$ HIGH = 1

Step 4: LOW=2 > HIGH=1 $\rightarrow$ NOT FOUND $\rightarrow$ return -1

Case	Time Complexity
Best Case	$O(1)$ — Key is at the middle
Average Case	$O(\log n)$
Worst Case	$O(\log n)$ — Key not present

□ **Key Point:** Binary Search is much faster than Linear Search for large sorted arrays. For $n=1,000,000$ elements, Linear takes up to 1,000,000 steps, Binary takes at most 20 steps!

8.4 Linear vs Binary Search

Feature	Linear Search	Binary Search
Data Requirement	Works on any array	Array must be sorted
Time Complexity	$O(n)$	$O(\log n)$
Comparisons	Up to n comparisons	At most $\log_2(n)$ comparisons
Implementation	Simple	Slightly complex
Best Use	Small or unsorted data	Large sorted datasets

9. Quick Revision Summary

9.1 Key Terminologies

Term	Meaning
Sorting	Rearranging elements in a specific order (ascending/descending)
Searching	Finding a specific element (key) in a data collection
Pivot	The selected reference element used in Quick Sort partitioning
Pass	One complete scan through the unsorted portion of the array
Stable Sort	Preserves the relative order of equal elements
In-place Sort	Uses $O(1)$ extra space — modifies the original array
Divide & Conquer	Strategy of breaking problem into sub-problems (Quick Sort, Merge Sort)
Linear Search	Sequential element-by-element search — $O(n)$
Binary Search	Halving-based search on sorted array — $O(\log n)$
Overflow	No room to insert (relevant in array-based structures)

9.2 Practice Questions

Important Questions

- What is sorting? Explain the types of sorting algorithms with examples.
- Trace Bubble Sort on: 64, 34, 25, 12, 22, 11, 90. Show all passes.
- Write and explain the Selection Sort algorithm with C code.

- Differentiate between Bubble Sort, Selection Sort, and Insertion Sort.
- Explain Quick Sort with the partition algorithm. Trace on: 7, 12, 1, -2, 0, 15, 4, 11, 9.
- Draw the Merge Sort tree for: 38, 27, 43, 3, 9, 82, 10.
- Compare all sorting algorithms based on time and space complexity.
- Write and explain Linear Search algorithm with C code.
- Trace Binary Search for key=40 in: 10, 20, 30, 40, 50, 60, 70.
- When would you prefer Binary Search over Linear Search? Justify with complexity analysis.

10. References & Resources

10.1 Text Books

- "Data Structure: A Pseudo code approach with C" — Richard F. Gilberg & Behrouz A. Forouzan, Thomson Publication, 2nd Ed.
- "Data Structure in C" — A.M. Tanenbaum, Y. Langsam, M.J. Augenstein, PHI/Pearson, 7th Ed.

10.2 Reference Books

- "Data Structures with C" (Schaum's Series) — Seymour Lipschutz, Tata-McGraw-Hill, 1st Ed.
- "Fundamentals of Data Structure in C" — Horowitz, Sahani & Freed, Computer Science Press, 2nd Ed.
- "Data Structures Using C" — Reema Thareja, Oxford University Press, 2nd Ed.

10.3 Online Sources

- <https://nptel.ac.in/courses/106104128>
- <https://www.geeksforgeeks.org/sorting-algorithms/>
- <https://www.javatpoint.com/binary-search>
- <https://btechx.infinityfreeapp.com>

— End of Unit V Notes —

BTechX